COMP 018: Database Administration - Activity 5 Progress Report
Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: HabalGo: Ride and Booking Management System
Target Project Group	: Group 6 (Royo, Villareal, Barte, Balansay, Remoquillo, Gumahin)
Target Project Repository	: https://github.com/jeanaroyo/HabalGo
Primary Database	: grp00_habalgodb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Target Schema	: habalgo
Restored Test Database	: grp00_habalgodb_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection
Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 8.66 megabytes across eight base tables and seven sequences within the habalgo schema. Table storage analysis showed that the booking table represents the largest relation at 72 kilobytes, followed by rating feedback at 64 kilobytes, and payment, driver, vehicle, and customer tables at 40 kilobytes each. Driver-vehicle assignment and location tables account for 24 kilobytes each.

2.2 Connection, Session & Activity Monitoring
Database metrics indicated one active connection with 109 committed transactions and 13 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across sessions, confirming stable transaction processing. Transaction rollbacks reflected permission denial tests conducted during Activity 4 verification. Session activity monitoring confirmed that only administrative connection threads were active, with no long-running or stalled connections.

2.3 Table Statistics & Bloat Inspection
Pre-maintenance table statistics revealed dead tuple accumulation across active tables, including 17 dead tuples in the driver table, 8 in the vehicle table, 7 in driver-vehicle assignments, and 1 in booking records resulting from initial testing and sequence resets. System catalog statistics showed null values for recent maintenance timestamps, confirming the necessity of performing tuple cleanup and planner statistics updates.

2.4 Log Review & Issue Identification
Review of database engine logs verified that unauthorized access attempts during Activity 4

security testing were recorded by the engine. Logged events included initial role creation conflicts, permission denials on record deletion and update commands, rejected database creation requests, and blocked cross-database connections. Server background processes completed without operational errors or crashes.

2.5 Routine Maintenance Operations
Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Permission boundary checks confirmed that application runtime accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative table ownership credentials.

Standard vacuuming and statistics updating executed under administrative credentials scanned target relations, removing 17 dead tuples from the driver table, 8 from the vehicle table, 7 from driver-vehicle assignments, and 1 from the booking table. Post-maintenance verification confirmed that dead tuple counts were reduced to zero across all tables, full page efficiency was restored, and planner statistics were updated.

Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application read and write operations. Because all relations reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples without causing application downtime.

2.6 Query Performance Analysis
Performance analysis was conducted on a multi-table join query retrieving booking details, customer names, pickup and dropoff locations, driver names, and booking statuses. The query executed in 0.963 milliseconds with complete buffer cache hit efficiency. The query engine utilized chained in-memory hash joins across booking, location, driver, and customer tables, followed by an in-memory quicksort, selecting sequential scans over index scans due to small memory page sizes.

Analysis of a second dashboard aggregation query calculating customer counts, active bookings, total revenue, and average ratings yielded an execution time of 0.426 milliseconds with zero physical disk reads. The execution plan processed four independent sub-plans in memory, consolidating scalar values into a single result row.

2.7 Prioritized Recommendations & Technical Justification
The existing index configuration is adequate for current operational needs, as queries execute under 1.0 millisecond with full buffer cache hits. Immediate index additions are unnecessary and would introduce write overhead during booking insertions. If platform transaction volume increases significantly in production, creating a partial composite index covering active booking statuses will optimize dashboard query processing. Additionally, deploying an automated weekly background maintenance task will maintain tuple efficiency during periods of low activity.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	**Value**
Project	Group 6 - HabalGo: Ride and Booking Management System
Database inspected	grp00_habalgodb (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), habalgo_app_user (Runtime), habalgo_readonly_user (Audit)
Health check result	Healthy (0 conflicts, 0 deadlocks, 8.66 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on driver, vehicle, driver_vehicle, booking via postgres
Query analyzed	Multi-table booking join query (Hash joins, 0.963 ms, 8 buffer hits) & Dashboard aggregate query (0.426 ms)
Recommendation priority	No action required (Current queries execute in < 1 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity5_Progress_Report_Group6.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.